An algorithm for Aubert–Zelevinsky duality

Thomas Lanard
(joint work with Alberto Mínguez)

Zagreb, November 17 - 21, 2025

Workshop Representation theory of p-adic groups

Get the slides:

https://www.thomaslanard.com/slides/AZ.pdf

An involution for finite reductive groups

Unipotent characters of $\mathrm{GL}_n(\mathbb{F}_q)$ are parametrized by partitions of n.

There is a natural involution given by transpose of partitions.

Theorem (Alvis–Curtis duality)

This involution is the Alvis–Curtis duality for $\mathbf{GL}_n(\mathbb{F}_q)$.

An involution for finite reductive groups

Unipotent characters of $\mathrm{GL}_n(\mathbb{F}_q)$ are parametrized by partitions of n.

There is a natural involution given by transpose of partitions.

Theorem (Alvis–Curtis duality)

This involution is the Alvis–Curtis duality for $\mathrm{GL}_n(\mathbb{F}_q)$.

$\mathrm{GL}_n(\mathbb{Q}_p)$		$\mathrm{GL}_n(\mathbb{F}_q)$
Aubert–Zelevinsky	$\longleftrightarrow$	Alvis–Curtis
Multisegments	$\longleftrightarrow$	Partitions
Mœglin–Waldspurger	$\longleftrightarrow$	Transpose

Aubert–Zelevinsky involution

Notations

Setting:

- F a p-adic field with norm $|\cdot|$
- G a p-adic group

Representations:

- $\mathrm{Rep}(G)$ = smooth complex finite-length representations
- $\mathrm{Irr}(G)$ = irreducible representations
- $\mathrm{Cusp}(G)$ = irreducible cuspidal representations

Functors: For a parabolic subgroup $P = MN$,

$$\mathrm{i}_P^G : \mathrm{Rep}(M) \to \mathrm{Rep}(G), \quad \mathrm{r}_P^G : \mathrm{Rep}(G) \to \mathrm{Rep}(M)$$

(normalized parabolic induction and restriction).

Operator on the Grothendieck group

Let $\mathrm{R}(G)$ be the Grothendieck group of $\mathrm{Rep}(G)$.

Definition

Define the operator

$$\mathrm{D_G}\colon\ \mathrm{R}(G) \longrightarrow \mathrm{R}(G)$$

by

$$\mathrm{D_G}(\pi) := \sum_{\substack{P=MU \\ \text{standard}}} (-1)^{\dim(A_M)}\ \mathrm{i}_{\mathrm{P}}^{\mathrm{G}}\big(\mathrm{r}_{\mathrm{P}}^{\mathrm{G}}(\pi)\big)$$

where A_M is the maximal split torus in the center of M.

The Aubert–Zelevinsky involution

Theorem (Aubert 1995)

If $\pi \in \mathrm{Irr}(G)$, there exists $\varepsilon \in \{\pm 1\}$ such that

$$\widehat{\pi} := \varepsilon \, \mathrm{D}_G(\pi) \in \mathrm{Irr}(G).$$

This defines a map, called the Aubert–Zelevinsky involution:

$$\begin{array}{ccc} \mathrm{Irr}(G) & \longrightarrow & \mathrm{Irr}(G) \\ \pi & \longmapsto & \widehat{\pi} \end{array}$$

Key properties

- The map $\pi \mapsto \widehat{\pi}$ is an involution on $\mathrm{Irr}(G)$.

Key properties

- The map $\pi \mapsto \widehat{\pi}$ is an involution on $\mathrm{Irr}(G)$.
- If π is cuspidal, then $\widehat{\pi} = \pi$.

Key properties

- The map $\pi \mapsto \widehat{\pi}$ is an involution on $\mathrm{Irr}(G)$.
- If π is cuspidal, then $\widehat{\pi} = \pi$.
- It exchanges the trivial and the Steinberg.

$$\widehat{\mathbf{1}_G} = \mathrm{St}_G \quad \text{and} \quad \widehat{\mathrm{St}_G} = \mathbf{1}_G$$

The case of GL_n

Parabolic induction for GL_n

We denote by $\times$ the parabolic induction for GL_n.

If $\pi_i \in \mathrm{Rep}(\mathrm{GL}_{n_i})$, then

$$\pi_1 \times \pi_2 = i_{\mathrm{GL}_{n_1} \times \mathrm{GL}_{n_2}}^{\mathrm{GL}_{n_1+n_2}}(\pi_1 \boxtimes \pi_2).$$

We write $\mathrm{Cusp}(\mathrm{GL}) = \bigsqcup_{n \geq 1} \mathrm{Cusp}(\mathrm{GL}_n)$ for the set of all cuspidal representations.

Segments

Definition

A segment is a subset of $\mathrm{Cusp}(\mathrm{GL})$ of the form

$$\Delta = \{\rho|\cdot|^x, \rho|\cdot|^{x+1}, \dots, \rho|\cdot|^y\}$$

where $\rho \in \mathrm{Cusp}(\mathrm{GL})$, and $x \leq y$ with $y - x \in \mathbb{N}$.

We use the notation $\Delta = [x, y]_\rho$.

Example: $\Delta = [0, 2]_\rho = \{\rho, \rho|\cdot|, \rho|\cdot|^2\}$

Operations on segments

For a segment $\Delta = [x, y]_\rho$ (with ρ unitary):

- $e(\Delta) = y$ (the end of Δ)
- $b(\Delta) = x$ (the beginning of Δ)
- $c(\Delta) = \frac{x+y}{2}$ (the center of Δ)

Let $\Delta^- = [x, y-1]_\rho$ (removing the last element)

Example: For $\Delta = [0, 2]_\rho$:

- $e(\Delta) = 2$, $b(\Delta) = 0$, $c(\Delta) = 1$
- $\Delta^- = [0, 1]_\rho = \{\rho, \rho|\cdot|\}$

Multisegments

Definition

A multisegment is a finite formal sum of segments:

$$\mathfrak{m} = \Delta_1 + \cdots + \Delta_r.$$

We denote by Mult the set of all multisegments.

Example: $\mathfrak{m} = [2,3]_\rho + [2,3]_\rho + [0,2]_\rho + [2,2]_\rho$ is a multisegment.

Zelevinsky classification

Theorem (Zelevinsky 1980)

To every segment $\Delta = [x, y]_\rho$, we can associate two irreducible representations $Z(\Delta)$ and $L(\Delta)$:

- $Z(\Delta)$ is the unique irreducible subrepresentation of the induced representation
$$\rho|\cdot|^x \times \cdots \times \rho|\cdot|^y$$
- $L(\Delta)$ is the unique irreducible quotient of this induced representation.

Zelevinsky classification

Theorem (Zelevinsky 1980)

To every multisegment $\mathfrak{m} = \Delta_1 + \cdots + \Delta_r$, we associate:

- $Z(\mathfrak{m})$: unique irreducible subrepresentation of $Z(\Delta_1) \times \cdots \times Z(\Delta_r)$
- $L(\mathfrak{m})$: unique irreducible quotient of $L(\Delta_1) \times \cdots \times L(\Delta_r)$

(satisfying certain technical conditions).

The maps $\mathfrak{m} \mapsto Z(\mathfrak{m})$ and $\mathfrak{m} \mapsto L(\mathfrak{m})$ give bijections

$$\mathrm{Mult} \xrightarrow{\sim} \mathrm{Irr}(\mathrm{GL}).$$

Zelevinsky classification

Theorem (Zelevinsky 1980)

To every multisegment $\mathfrak{m} = \Delta_1 + \cdots + \Delta_r$, we associate:

- $Z(\mathfrak{m})$: unique irreducible subrepresentation of $Z(\Delta_1) \times \cdots \times Z(\Delta_r)$
- $L(\mathfrak{m})$: unique irreducible quotient of $L(\Delta_1) \times \cdots \times L(\Delta_r)$

(satisfying certain technical conditions).

The maps $\mathfrak{m} \mapsto Z(\mathfrak{m})$ and $\mathfrak{m} \mapsto L(\mathfrak{m})$ give bijections

$$\mathrm{Mult} \xrightarrow{\sim} \mathrm{Irr}(\mathrm{GL}).$$

Fact: $\widehat{Z(\mathfrak{m})} = L(\mathfrak{m})$

The duality problem for GL_n

Question

Given a multisegment $\mathfrak{m}$, find $\mathfrak{m}^t$ such that

$$\widehat{Z(\mathfrak{m})} = Z(\mathfrak{m}^t).$$

An explicit algorithm was given by Mœglin and Waldspurger.

Reduction to cuspidal lines

The algorithm works on each cuspidal line independently.

Definition

A cuspidal line is

$$\mathbb{Z}_\rho = \{\rho|\cdot|^n,\ n \in \mathbb{Z}\}$$

for a fixed cuspidal representation $\rho \in \mathrm{Cusp}(\mathrm{GL})$.

Let Mult_ρ = multisegments supported on $\mathbb{Z}_\rho$.

Then $\mathrm{Mult} = \bigoplus_\rho \mathrm{Mult}_\rho$ (where the sum runs over cuspidal lines), and this decomposition is compatible with duality:

$$\mathfrak{m} = \sum_\rho \mathfrak{m}_\rho \quad \Longrightarrow \quad \mathfrak{m}^t = \sum_\rho \mathfrak{m}_\rho^t$$

Order on segments

Definition (Order on segments)

For segments on the same cuspidal line:

$$[x,y]_\rho \leq [x',y']_\rho \quad \text{if} \quad \begin{cases} x < x' \\ \text{or } x = x' \text{ and } y \geq y' \end{cases}$$

⚠ This is NOT the lexicographic order!

Example: $[0,2]_\rho \leq [1,6]_\rho \leq [1,3]_\rho$

The Mœglin–Waldspurger algorithm

Let $\mathfrak{m} \in \mathrm{Mult}_\rho$ be a multisegment.

Step 1: Find $e_{\max} = \max\{e(\Delta) \mid \Delta \in \mathfrak{m}\}$.

Step 2: Let Δ_1 be the largest segment (for $\leq$) with $e(\Delta_1) = e_{\max}$.

Step 3: Define inductively $\Delta_1 \geq \Delta_2 \geq \cdots \geq \Delta_l$ where Δ_{j+1} is the largest segment such that:

- $\Delta_{j+1} \leq \Delta_j$
- $e(\Delta_{j+1}) = e(\Delta_j) - 1$

(we stop if no such segment exist)

The Mœglin–Waldspurger algorithm

Having found $\Delta_1, \ldots, \Delta_l$, define:

- The first segment of the dual:

$$\Delta_1' = [e(\Delta_l), e(\Delta_1)]_\rho$$

- A reduced multisegment:

$$\mathfrak{m}^\# = \mathfrak{m} - \sum_{j=1}^{l} (\Delta_j - \Delta_j^-)$$

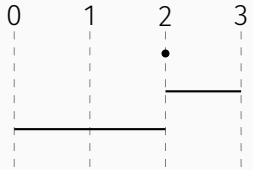
Theorem (Mœglin–Waldspurger 1986)

The dual multisegment is given recursively by:

$$\mathfrak{m}^t = \Delta_1' + (\mathfrak{m}^\#)^t$$

Example 1

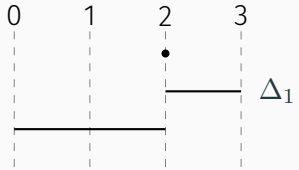
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = \cdots$

Example 1

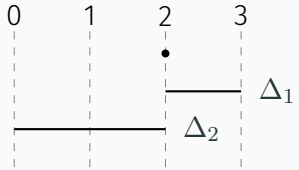
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = \cdots$

Example 1

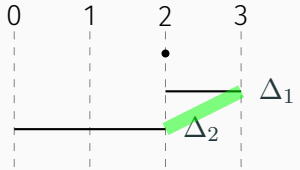
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = \cdots$

Example 1

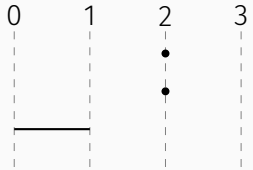
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + \cdots$

Example 1

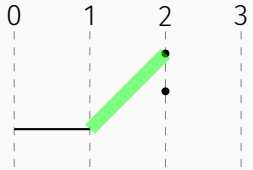
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + \cdots$

Example 1

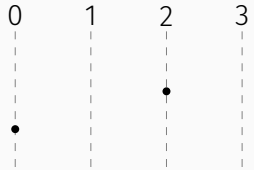
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + [1,2] + \cdots$

Example 1

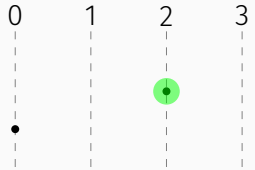
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + [1,2] + \cdots$

Example 1

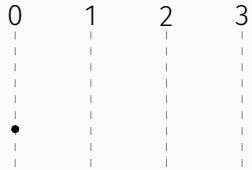
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + [1,2] + [2,2] + \cdots$

Example 1

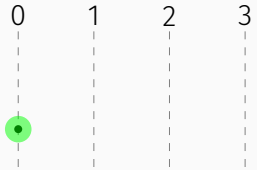
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + [1,2] + [2,2] + \cdots$

Example 1

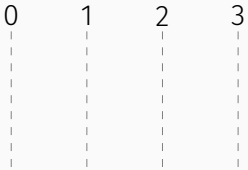
Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + [1,2] + [2,2] + [0,0]$

Example 1

Compute the dual of $\mathfrak{m} = [2,3] + [0,2] + [2,2]$.



Dual multisegment: $\mathfrak{m}^t = [2,3] + [1,2] + [2,2] + [0,0]$

Example 2: Transpose of a partition

Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:

Example 2: Transpose of a partition

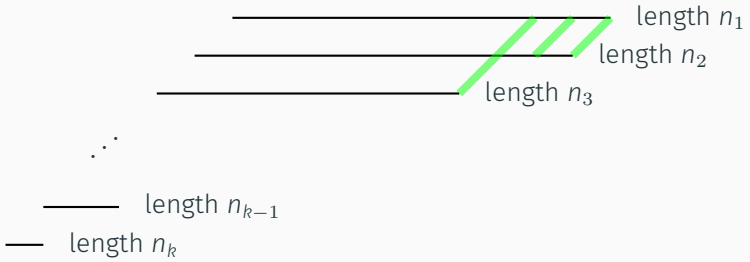
Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:

Example 2: Transpose of a partition

Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:

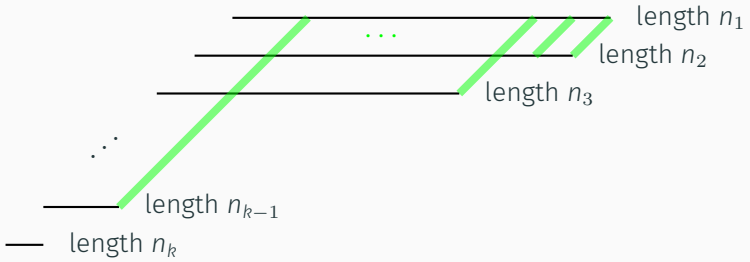
Example 2: Transpose of a partition

Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:



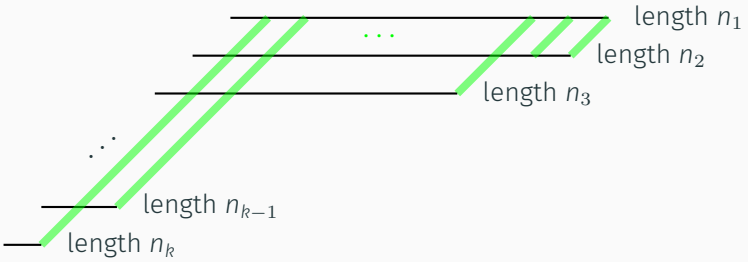
Example 2: Transpose of a partition

Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:



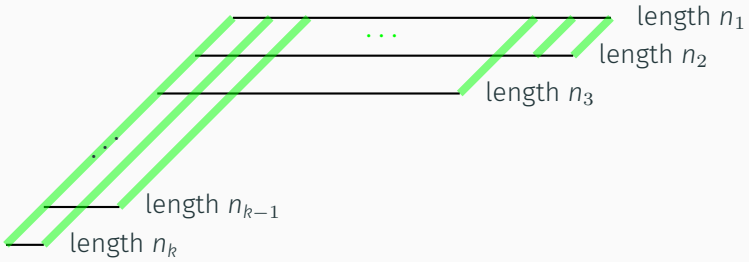
Example 2: Transpose of a partition

Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:



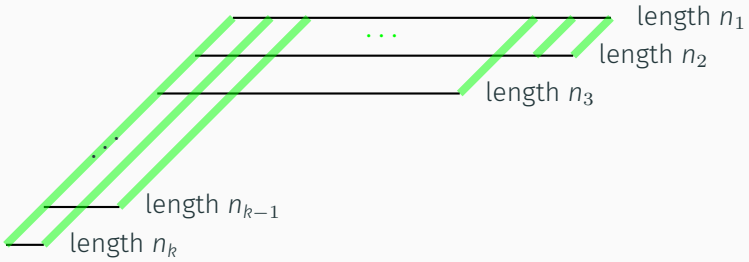
Example 2: Transpose of a partition

Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:



Example 2: Transpose of a partition

Let $n_1 \geq \cdots \geq n_k$ be a partition. Consider the multisegment $\mathfrak{m}$ with segments of lengths $n_1, \ldots, n_k$ and starting points decreasing by 1:



Then $\mathfrak{m}^t$ corresponds to the transpose partition (n_i^*).

The case of classical groups

Classification of irreducible representations

We now consider $G = \mathrm{Sp}_{2n}(F)$ or $\mathrm{SO}_{2n+1}(F)$.

Classification via local Langlands correspondence

Irreducible representations of G are parametrized by Langlands data

$$\pi = L(\mathfrak{m}, \phi, \eta)$$

Components of the Langlands data:

- **Multisegment:** $\mathfrak{m} = [x_1, y_1]_{\tau_1} + \cdots + [x_r, y_r]_{\tau_r}$, where $x_i + y_i < 0$
- **Langlands parameter:** $\phi = \bigoplus_{i=1}^{k} \rho_i \boxtimes S_{a_i}$, where $\rho_i \in \mathrm{Cusp}(\mathrm{GL})$ and $a_i \in \mathbb{N}^*$
- **Character:** η assigns signs ± 1 to each $\rho_i \boxtimes S_{a_i}$

(satisfying technical conditions)

Symmetrization of the data

- For $\Delta = [x, y]_\rho$, define $\Delta^\vee = [-y, -x]_{\rho^\vee}$
- By linearity, it extends to Mult

Example: For $\mathfrak{m} = [-1, 2] + [-1, 1]$ we have $\mathfrak{m}^\vee = [-2, 1] + [-1, 1]$

Let $\mathrm{Symm} = \{\mathfrak{s} \in \mathrm{Mult} \mid \mathfrak{s}^\vee = \mathfrak{s}\}$ (symmetric multisegments).

Define $\mathrm{Symm}^\varepsilon$ (symmetric multisegments with signs) as pairs $(\mathfrak{s}, \varepsilon)$ where:

- $\mathfrak{s} \in \mathrm{Symm}$
- ε assigns signs ± 1 to centered segments (those with $c(\Delta) = 0$)

The transfer map

From Langlands data to symmetric multisegment:

$$\begin{aligned} \mathrm{Data}(G) &\longrightarrow \mathrm{Symm}^{\varepsilon} \\ (\mathfrak{m}, \phi, \eta) &\longmapsto (\mathfrak{s}, \varepsilon) \end{aligned}$$

where the symmetric multisegment is

$$\mathfrak{s} = \sum_{\Delta \in \mathfrak{m}} (\Delta + \Delta^{\vee}) + \sum_{\rho \boxtimes S_a \in \phi} \left[-\frac{a-1}{2}, \frac{a-1}{2} \right]_{\rho}$$

and the signs are given by $\varepsilon\left(\left[-\frac{a-1}{2}, \frac{a-1}{2}\right]_{\rho}\right) = \eta(\rho \boxtimes S_a)$

We write $\pi = L(\mathfrak{s}, \varepsilon)$ for this symmetrized form.

The AD algorithm

We define an involution

$$\mathrm{AD} : \mathrm{Symm}^{\varepsilon} \longrightarrow \mathrm{Symm}^{\varepsilon}$$

Theorem (L.–Mínguez)

Let $\pi = L(\mathfrak{s}, \varepsilon) \in \mathrm{Irr}(G)$. Then

$$\widehat{\pi} = L(\mathrm{AD}(\mathfrak{s}, \varepsilon)).$$

The AD algorithm

We define an involution

$$\mathrm{AD} : \mathrm{Symm}^{\varepsilon} \longrightarrow \mathrm{Symm}^{\varepsilon}$$

Theorem (L.–Mínguez)

Let $\pi = L(\mathfrak{s}, \varepsilon) \in \mathrm{Irr}(G)$. Then

$$\widehat{\pi} = L(\mathrm{AD}(\mathfrak{s}, \varepsilon)).$$

Remark: It is not easy to find an involution on $\mathrm{Symm}^{\varepsilon}$!

Three types of cuspidal representations

- AD is defined on each cuspidal line $\mathbb{Z}_\rho$
- Unlike GL_n, the cuspidal ρ matters!
- Three cases depending on ρ: ugly, bad, good

Structure of the AD algorithm

AD is defined recursively:

For $(\mathfrak{s}, \varepsilon) \in \mathrm{Symm}^{\varepsilon}$, construct:

- $(\mathfrak{s}_1, \varepsilon_1)$: first piece of the dual
- $(\mathfrak{s}^{\#}, \varepsilon^{\#})$: reduced symmetric multisegment

such that

$$\mathrm{AD}(\mathfrak{s}, \varepsilon) = (\mathfrak{s}_1, \varepsilon_1) + \mathrm{AD}(\mathfrak{s}^{\#}, \varepsilon^{\#})$$

AD algorithm (good case)

Let $(\mathfrak{s}, \varepsilon) \in \mathrm{Symm}^{\varepsilon}$.

Step 1: Order segments by a specific order $\preceq$.

Step 2: Construct descending chain $\Delta_1 \succeq \cdots \succeq \Delta_r$ where:

- Δ_1 is the largest segment with maximal end
- Δ_{j+1} is the largest segment such that:
  1. $\Delta_{j+1} \preceq \Delta_j$
  2. $e(\Delta_{j+1}) = e(\Delta_j) - 1$
  3. Sign condition: if $c(\Delta_{j+1}) = c(\Delta_j) = 0$ then $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Step 3: Extract $\mathfrak{s}_1$ from ends of Δ_j and beginnings of $\Delta_j^{\vee}$.

Step 4: Compute $\mathfrak{s}^{\#}$ by removing these parts + adjust signs.

Example

Setting: $G = \mathrm{Sp}_{2n}(F)$ and $\rho = 1$ (good case)

Langlands datum:

$$\pi = L([-2,0] + [-1,-1] + [-1,-1],\ S_1 + S_1 + S_3,\ (-1,-1,1))$$

Example

Setting: $G = \mathrm{Sp}_{2n}(F)$ and $\rho = 1$ (good case)

Langlands datum:

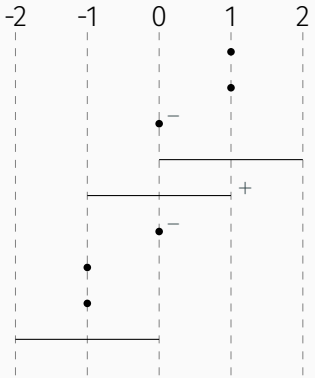
$$\pi = L([-2,0] + [-1,-1] + [-1,-1],\ S_1 + S_1 + S_3,\ (-1,-1,1))$$

Symmetrization:

$$\mathfrak{s} = [-2,0] + [0,2] + [-1,-1] + [1,1] + [-1,-1] + [1,1] + [0,0] + [0,0] + [-1,1]$$

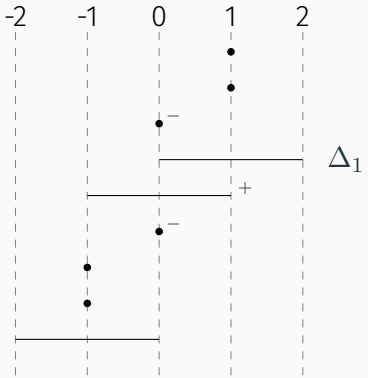
with $\varepsilon([0,0]) = -1$ and $\varepsilon([-1,1]) = 1$.

Example: Step by step



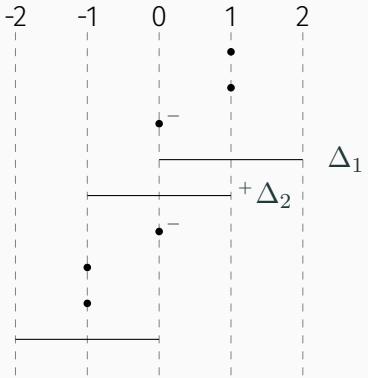
Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = \cdots$

Example: Step by step



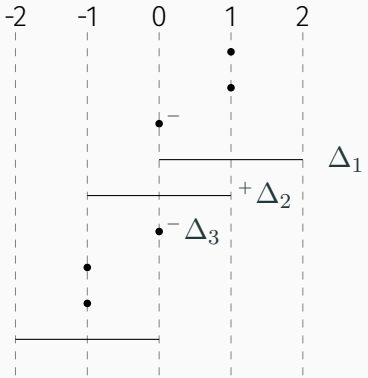
Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = \cdots$

Example: Step by step



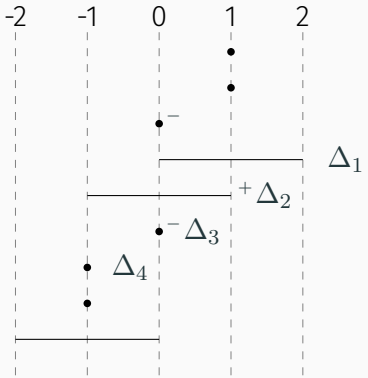
Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = \cdots$

Example: Step by step



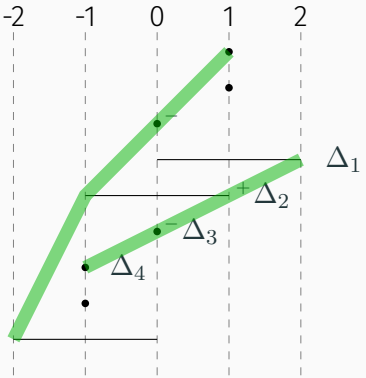
Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = \cdots$

Example: Step by step



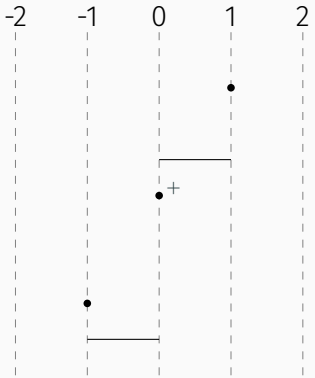
Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = \cdots$

Example: Step by step



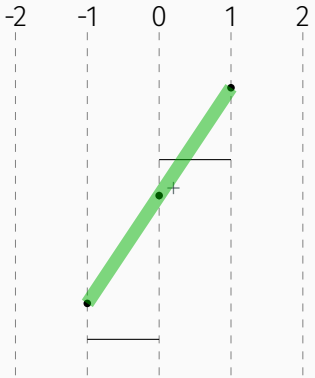
Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = [-1, 2] + [-2, 1] + \cdots$

Example: Step by step



Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = [-1, 2] + [-2, 1] + \cdots$

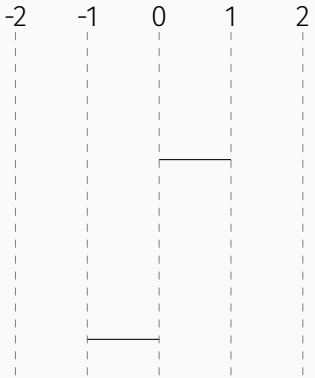
Example: Step by step



Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = [-1, 2] + [-2, 1] + [-1, 1] + \cdots$

with $\varepsilon([-1, 1]) = 1$

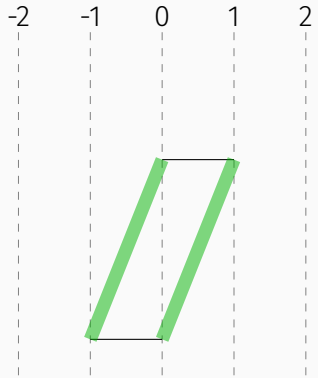
Example: Step by step



Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = [-1, 2] + [-2, 1] + [-1, 1] + \cdots$

with $\varepsilon([-1, 1]) = 1$

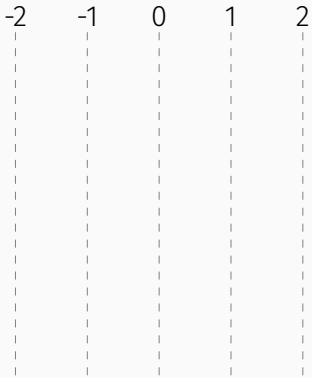
Example: Step by step



Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = [-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0]$

with $\varepsilon([-1, 1]) = 1$

Example: Step by step



Dual: $\mathrm{AD}(\mathfrak{s}, \varepsilon) = [-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0]$

with $\varepsilon([-1, 1]) = 1$

Example: Result

Input:

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Example: Result

Input:

$$\pi = L([-2,0]+[-1,-1]+[-1,-1], S_1+S_1+S_3, (-1,-1,1))$$

Symmetrize + apply AD:

$$\mathrm{AD}(\mathfrak{s},\varepsilon) = ([-1,2]+[-2,1]+[-1,1]+[0,1]+[-1,0], \varepsilon')$$

with $\varepsilon'([-1,1]) = 1$.

Example: Result

Input:

$$\pi = L([-2,0]+[-1,-1]+[-1,-1], S_1+S_1+S_3, (-1,-1,1))$$

Symmetrize + apply AD:

$$\mathrm{AD}(\mathfrak{s},\varepsilon) = ([-1,2]+[-2,1]+[-1,1]+[0,1]+[-1,0], \varepsilon')$$

with $\varepsilon'([-1,1]) = 1$.

Converting back to Langlands data:

$$\widehat{\pi} = L([-2,1]+[-1,0], S_3, (1))$$

Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.

Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.

$\mathfrak{m} = [-2, 0] + 2[-1, -1]$
$\phi = S_1 + S_1 + S_3$
$\eta = (-1, -1, 1)$

Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.

$$\begin{array}{ll} \mathfrak{m} = [-2,0] + 2[-1,-1] & \\ \phi = S_1 + S_1 + S_3 & \longrightarrow \\ \eta = (-1,-1,1) & \end{array} \begin{array}{l} D_{|\cdot|^{-1}} \\ \\ \\ \end{array} \begin{array}{l} \mathfrak{m} = [0,-2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array}$$

Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.

$$\begin{array}{l} \mathfrak{m} = [-2,0] + 2[-1,-1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{|\cdot|^{-1}}} \begin{array}{l} \mathfrak{m} = [0,-2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{L([-1,0])}} \begin{array}{l} \mathfrak{m} = [-2,-2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array}$$

Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.

$$\begin{array}{l} \mathfrak{m} = [-2,0] + 2[-1,-1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{|\cdot|^{-1}}} \begin{array}{l} \mathfrak{m} = [0,-2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{L([-1,0])}} \begin{array}{l} \mathfrak{m} = [-2,-2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{|\cdot|^{-2}}} \begin{array}{l} \mathfrak{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array}$$

Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.

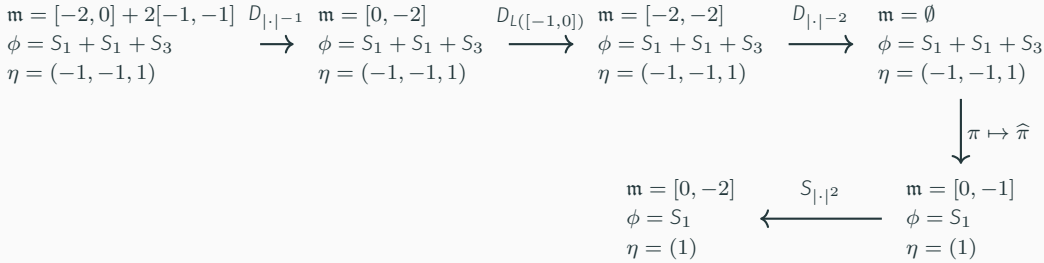
$$\begin{array}{l} \mathfrak{m} = [-2,0] + 2[-1,-1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{|\cdot|^{-1}}} \begin{array}{l} \mathfrak{m} = [0,-2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{L([-1,0])}} \begin{array}{l} \mathfrak{m} = [-2,-2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array} \xrightarrow{D_{|\cdot|^{-2}}} \begin{array}{l} \mathfrak{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1,-1,1) \end{array}$$

$$\downarrow \pi \mapsto \widehat{\pi}$$

$$\begin{array}{l} \mathfrak{m} = [0,-1] \\ \phi = S_1 \\ \eta = (1) \end{array}$$

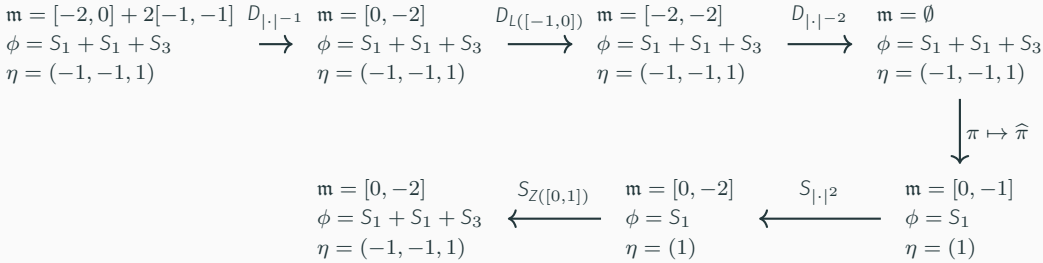
Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.



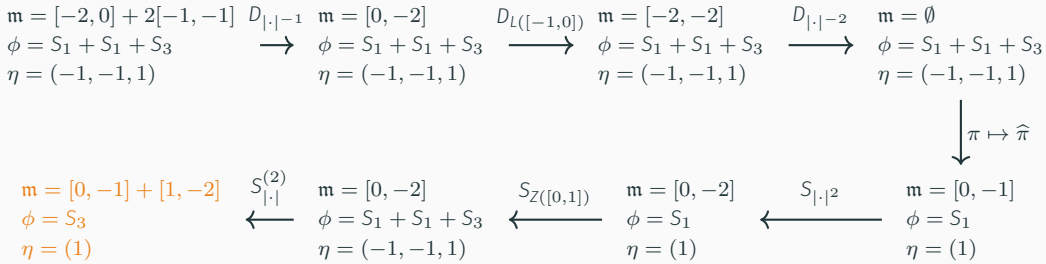
Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.



Comparison with Atobe–Mínguez algorithm

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ-derivatives.



Implementation

Implementation

Implementations:

- Python/Sage package:
  `github.com/ThomasLanard/aubert-zelevinsky-duality`
- Web application (in development with Petar Bakić and Elad Zelingher):
  `langlandsprograms.com`

  Computational tools for representation theory and the Langlands program.

Machine learning

Machine learning approach

Motivation:

- AI excels at finding patterns in large datasets.
- In mathematics, generating large and precise datasets is straightforward.
- AI can be used to uncover new relations between mathematical objects.

Machine learning approach

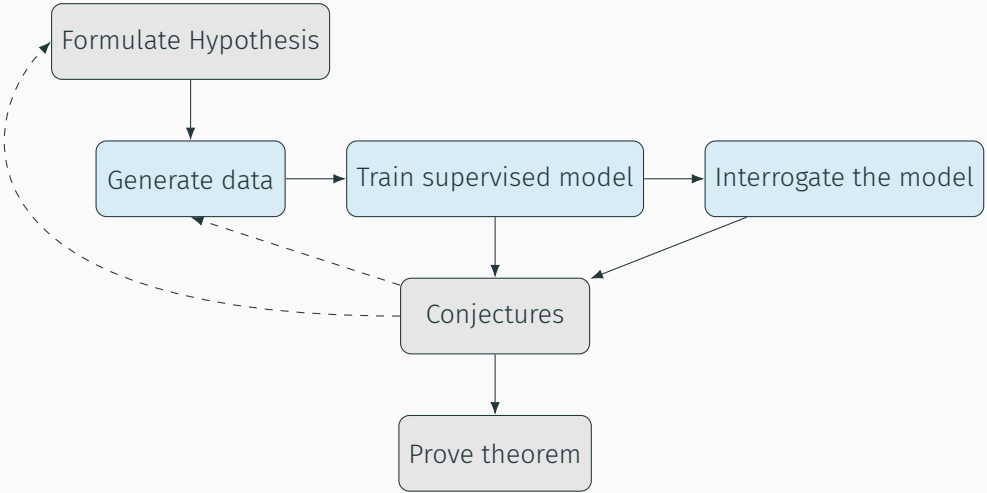
Motivation:

- AI excels at finding patterns in large datasets.
- In mathematics, generating large and precise datasets is straightforward.
- AI can be used to uncover new relations between mathematical objects.

In our work:

- We use supervised learning, relying on the Atobe–Mínguez algorithm, to gain intuition about the Aubert–Zelevinsky duality.
- We began this process without making any mathematical assumptions or conjectures, in order to explore what structures the machine learning model might reveal on its own.

Machine learning workflow



Neural networks

A neural network is a function $\varphi : \mathbb{R}^n \to \mathbb{R}^m$ given by composition:

$$\varphi = \varphi_r \circ \cdots \circ \varphi_1$$

where each layer is:

$$\varphi_i : \mathbb{R}^{n_i} \xrightarrow{\text{linear}} \mathbb{R}^{n_{i+1}} \xrightarrow{\mathrm{ReLU}} \mathbb{R}^{n_{i+1}}$$

ReLU (Rectified Linear Unit): $x \mapsto \max(0, x)$ applied componentwise

(no ReLU on the last layer φ_r)

Supervised learning

Goal: Approximate an unknown function $\widehat{\varphi} : \mathbb{R}^n \to \mathbb{R}^m$

Training data: Samples $(x_i, \widehat{\varphi}(x_i))_{i=1,\dots,N}$

Loss function: Measures prediction error

$$l(\varphi) = \frac{1}{N} \sum_{i=1}^{N} \|\varphi(x_i) - \widehat{\varphi}(x_i)\|^2$$

Training: Use gradient descent to find parameters that minimize $l(\varphi)$

First attempt: Predict the entire dual

Question

Can we predict the entire dual $\mathrm{AD}(\mathfrak{m}, \phi, \eta)$ directly?

First attempt: Predict the entire dual

Question

Can we predict the entire dual $\mathrm{AD}(\mathfrak{m}, \phi, \eta)$ directly?

Result

The model shows poor accuracy.

First attempt: Predict the entire dual

Question

Can we predict the entire dual $\mathrm{AD}(\mathfrak{m}, \phi, \eta)$ directly?

Result

The model shows poor accuracy.

New strategy: Instead of predicting the entire dual at once, try to predict one segment.

Refined approach: Predict one segment

Question

Which segment should we predict first?

Thank you for your attention!